

Fizzle: A Framework for Deterministic and Reproducible Network Fuzzing

Nathaniel Bennett*, Tyler Tucker*, Carson Stillman*, William Enck[†], Patrick Traynor* and Kevin Butler*

*University of Florida, {bennett.n, tylertucker1, carson.stillman, traynor, butler}@ufl.edu

[†]North Carolina State University, whenck@ncsu.edu

Abstract—Networked systems regularly receive untrusted inputs. When improperly handled, adversaries thus have the ability to remotely crash or compromise such systems. While many techniques have been proposed to detect these vulnerabilities, their ability to handle asynchronous and nondeterministic behavior significantly limit their coverage of real systems. In this paper we present FIZZLE, a high-performance deterministic simulation testing framework for fuzzing network applications. FIZZLE interposes system library calls to enforce sequential execution for multi-threading, simulate multi-modal network connections, and control system randomization in a manner that is transparent to the target application. By tracking synchronization and I/O primitives, FIZZLE deterministically identifies when an application has finished processing input, thereby dramatically expanding the suite of protocols that it can accurately fuzz relative to other approaches. We show that FIZZLE maintains 100% stability across ProFuzzBench targets and highly concurrent applications including Unbound, Redis and Open5GS. This resultantly leads to improved coverage and newly discovered/disclosed CVEs for projects that have already been subject to fuzzing. Surprisingly, FIZZLE significantly outperforms existing network fuzzers *despite* carrying out substantial bookkeeping of external state and enforcing sequential execution of threads, achieving up to 20x more executions per second. FIZZLE is designed to be highly configurable to a variety of network inputs and integrates with both AFL++ and LibAFL; we release it as an open-source extensible framework for future network protocol research to be carried out in a deterministic and reproducible manner.

1. Introduction

Fuzzing is a crucial technique for assessing the security of real systems. Decades of work have culminated in modern fuzzers that can run across large and complex codebases, including networked code that stores significant amounts of state. However, while coverage-guided fuzzers are capable of generating complex and nuanced inputs for testing, their results can be both inefficient and difficult to reproduce because of system nondeterminism and target complexity. Such issues plague a wide variety of production-class software, making it a challenge to assess their threat surfaces.

In this paper, we respond to the fundamental challenges of stability faced by modern fuzzers and develop FIZZLE¹,

a new fuzzing solution tailored for large, complex, and networked systems. FIZZLE achieves its goals by addressing instability inherent in other approaches to fuzzing—it precisely tracks thread execution and models I/O and concurrency primitives called by the application, all the while replacing variable environment inputs with values derived from fuzzing input. This enables it to both enforce deterministic control flow and reliably identify the exact point where a server has finished executing fuzzing input without resorting to expensive timeouts. In so doing, FIZZLE improves on the performance of past approaches without requiring modifications to target source code while providing straightforward configuration for multi-input fuzzing. FIZZLE makes the following contributions:

- **Produce Stable and Bitwise Reproducible Fuzzing:** We observe existing network fuzzers suffer from the butterfly effect when even a small amount of instability is encountered – discovering crashes at wildly different times (or never) despite beginning with the same internal random seed. FIZZLE’s deterministic execution can enable fuzzing at 100% stability, resulting in bit-for-bit identical finds even after millions of executions.
- **Achieve Significantly Improved Coverage:** FIZZLE often achieves better coverage in an hour than other fuzzers achieve through a full 24-hour campaign, representing a 90% reduction in fuzzing time. Moreover, FIZZLE can attain substantially increased end coverage (sometimes as high as 3x) when compared to other fuzzers.
- **Significantly Higher Performance:** While serialization would seem to result in lower performance when compared to multithreading, we demonstrate that in-memory handling of synchronization and I/O primitives speeds up performance by as much as 20x due to reduced context-switching. Additionally, we show Persistent Channel Reset mode for particularly large/slow network applications sees 100x speed improvements.

FIZZLE additionally uncovers three new vulnerabilities in a system previously evaluated by novel network fuzzing techniques, two of which would be unreachable by any existing fuzzing framework. Though we focus our evaluation on network fuzzing, we note the broad applicability of FIZZLE as a deterministic execution environment for other dynamic analysis techniques. We design and release it [1] as a platform for further enabling deterministic runtime behavior.

1. So named after the effect of a *socket* being *shorted out*.

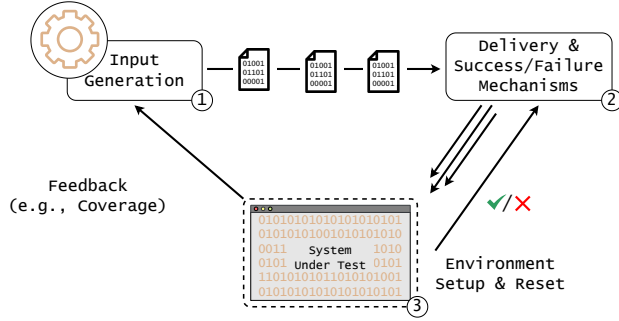


Figure 1. Three fundamental components in feedback-driven fuzz testing of a system.

While achieving deterministic runs helps in both reducing lost coverage feedback from instability and enabling reproducibility, the end goal of network fuzzing is to quickly explore a program’s control flow and identify input vulnerabilities. As such, our evaluation explores both the impacts of instability/determinism on other programs and FIZZLE respectively, and the coverage and speed impacts resulting from FIZZLE. We specifically consider the following five research questions:

- RQ1** Do existing mitigations to instability (e.g., calibration stages) impose a significant overhead on fuzzing performance?
- RQ2** Does environment emulation and interposition allow for more efficient exploration of the target program?
- RQ3** To what degree can the environment and execution of networked, multithreaded applications be made deterministic *without* fundamentally modifying their code?
- RQ4** How does program environment interposition/emulation impact execution speed for fuzzing?
- RQ5** Is FIZZLE capable of finding new vulnerabilities in target protocols?

We now cover background information relevant to FIZZLE (Section 2) and systematic challenges in network protocol fuzzing (Section 3) before describing the architecture of FIZZLE (Section 4). We then evaluate based on the research questions in Section 5, following which we provide discussion (Section 6) and related works (Section 7).

2. Background

Fuzzing Executors. Coverage-guided fuzzing can be organized into three fundamental components: a fuzzing engine, which generates sequential fuzzing inputs by queuing and mutating “interesting” seeds based on feedback; a mechanism by which fuzzing inputs are delivered to the system for interpretation and crashes are detected; and an environment in which the system is operated that resets the system after fuzzing rounds and passes feedback to the backend (see Figure 1). The latter two components are commonly considered together as the “Executor”; we distinguish the two to identify challenges unique to each component.

To fuzz effectively, a fuzzer needs to (a) know when to cut off execution of a current input, and (b) reset the state of the system back to a uniform starting point at the end of each fuzzing input. These tasks correspond directly to the second and third components mentioned above. For traditional forklift fuzzers, these challenges are trivial in nature—input is passed via `stdin`, so the close of an application constitutes the end of a fuzzing round. The forklift server then resets the application’s internal memory to a common starting copy of the application. Likewise, in-memory fuzzing of functions that don’t access external state (such as data decoding routines) is carried out by ending fuzzing rounds at the return of each function, and state is implicitly reset in each new function call. Such approaches are trivially easy primarily because the targets fuzzed by such approaches exhibit a linear, terminating control flow and do not access external state. Where these properties are violated—as is the case in network servers—they fundamentally affect both the rate at which inputs can be executed and the fidelity of coverage feedback.

Network Fuzzing. Whereas terminal utilities or conversion functions are designed to be run on a single input and terminate, network servers are designed from the ground up to operate continuously and execute asynchronously.

External State. Beyond input, networked systems additionally frequently access or mutate external state—often including calls to other network systems such as DNS—which introduce variance in the environment that have downstream effects application behavior for consecutive forks of the same application.

3. Challenges

Despite its status as an established method of discovering vulnerabilities, coverage-guided fuzzing still faces several challenges that limit its efficacy when applied to more complex target programs. We provide these challenges below and design FIZZLE to overcome each of them, as shown in Table 1.

3.1. Handling Instability During Coverage-Guided Feedback (Challenge 1)

Coverage-guided fuzzing uses feedback from a target program to select new mutated inputs for fuzzing. To achieve efficient coverage in this context, this technique requires that the target system behaves deterministically (i.e., coverage feedback is consistent) and resultantly exhibits high *stability*. If this is not the case, the fuzzer may add additional inputs to the scheduler that do not result in increased coverage, or else fail to add otherwise interesting inputs to the scheduler due to their coverage instability.

Current fuzzing engines attempt to mitigate this issue by testing for and permanently ignoring unstable feedback bits [9], [10]. To achieve this, the engine runs the same input several times and checks for inconsistent responses. Such checks are probabilistic by nature, so unstable bits will not

		AFLnet/SnapFuzz [2], [3]	NetDriver/SGFuzz [4], [5]	Preeny [6]	Nyx-Net [7]	Netfuzzlib [8]	Fizzle
Environment	Limitation	Prior Work					
Multithreaded Applications	Crash Misattribution	●	○	●	○	●	●
	Missed Crashes	○	○	○	○	○	●
	Coverage Misattribution	●	○	●	○	●	●
	Truncated Coverage	○	○	○	○	○	●
	Missed Responses	●	●	○	○	○	●
Non-reflexive Protocols	Crash Misattribution	●	○	●	●	●	●
	Missed Crashes	○	○	●	●	●	●
	Hang on Non-Response	○	○	●	●	●	●
	Truncated Coverage	○	○	○	○	○	●
	Coverage Misattribution	●	○	●	●	●	●
Forking Applications	No Support	●	○	●	○	○	●
Multi-modal Input	No Support	○	○	○	○	○	●
Nondeterminism	Degraded Stability	○	○	○	○	○	●

TABLE 1. COMPARISON OF PRIOR WORK WITH FIZZLE. ● INDICATES THAT A SYSTEM OVERCOMES THE RESPECTIVE CHALLENGE WHILE ○ INDICATES THE SYSTEM EXHIBITS THE GIVEN LIMITATION.

always be detected, but they offer some means of mitigating the drawbacks of nondeterminism at the cost of re-running identical inputs. The fuzzer then permanently ignores edges deemed to be unstable for the remainder of the campaign, thus preventing that portion of feedback from having any future use.

Figure 2 highlights the issues of stability in coverage-guided fuzzing. Mutated inputs that are passed to an application are executed and lead to coverage feedback being generated. Applications that exhibit nondeterminism in their execution trace (whether by multithreading, program environment inputs or external network dependencies) problematically output a potentially significant number of *unstable* edges. Such edges are either present or not present in repeated runs of the same fuzzing input and across consecutive distinct inputs. If a fuzzing engine does not account for the possibility of an unstable target, the nondeterministic edges lead to the scheduling of numerous fuzzing inputs that do not meaningfully explore new program behavior, but rather happened to trigger a new nondeterministic edge. The consequence of this approach is increased noise in the scheduler, as depicted in the lower branch of Figure 2.

Alternatively, a fuzzing engine that accounts for unstable targets will repeat a given input several times when new coverage feedback is found (generally referred to as *calibration*). If any edges are revealed to be inconsistent across these repeated tries, they are marked as being unstable and are not considered thereafter. The consequence of this approach is decreased signal to identify interesting inputs to the scheduler, as depicted in the upper branch of Figure 2.

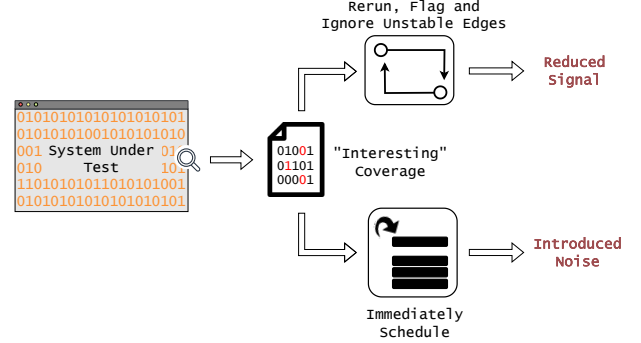


Figure 2. The effects of nondeterminism in coverage-guided greybox fuzzing. A process that uses nondeterministic mechanisms (such as multithreading, random entropy sources and timing APIs) emits differing coverage feedback for inputs that would result in identical execution behavior otherwise. Coverage-guided fuzzing engines are faced with the dilemma of either (a) scheduling these inputs, which leads to many seeds with identical behavior in the power scheduler taking up most of the work, or (b) re-testing inputs with new coverage to attempt to identify nondeterministic edges in the program and permanently ignore these.

While the latter approach is commonly adopted to make fuzzing of unstable targets less prone to instability, both approaches fundamentally exhibit reduced performance, particularly when a program exhibits a high degree of instability.

3.2. Fuzzing Networked Systems (Challenge 2)

Fuzzing frameworks commonly rely on the assumption that a *singular payload* may be passed into the system over a *well-defined interface* (such as standard input or a function parameter), and that the system will *gracefully exit or return* upon processing that input. Unfortunately, all of the above three assumptions are violated when it comes to fuzzing network server interfaces. First, network servers require sequential, stateful and precisely-sized inputs passed at sufficient time increments to ensure full handling is accomplished. Second, whereas function-based fuzzing may identify a certain decoding subroutine to fuzz, network servers handle inputs through series of callbacks or futures that frequently access and mutate global state. This inhibits any extraction of input subroutines to fuzz independantly of the rest of the system. Third, network servers do not terminate following the end of a connection, so the fuzzing framework is left using some heuristic to guess when the server might have fully finished processing all inputs.

The above challenges directly lead to the following undesired behaviors during fuzzing: (1) miss crashes that would occur after a response is sent (i.e., during post-processing of a packet in another thread), (2) miss responses completely, (3) mis-attribute crashes from one input round to another, (4) return unreliable coverage feedback even in the case of a program that would have otherwise behaved deterministically, and (5) cause expensive delays and timeouts when applications do not fully conform to the expectations for a given protocol.

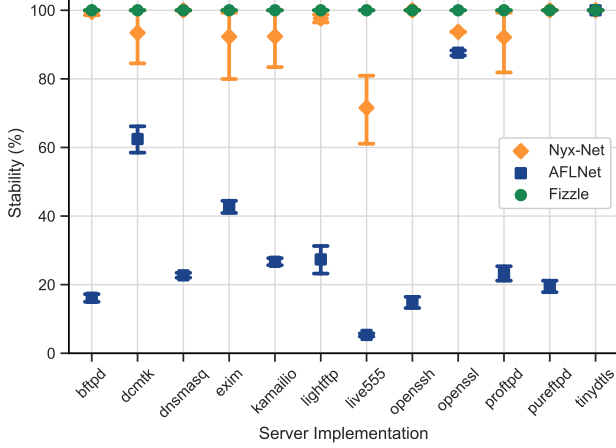


Figure 3. Stability scores for AFLNet, Nyx-Net and Fizzle fuzzing tools on the ProFuzzBench test suite over 24 hours. Error bars indicate 90% confidence interval across ten trials.

These behaviors lead to yet more instability in an application as coverage is missed or mis-attributed between consecutive runs. To quantify the effects of these behaviors, we run original artifacts from AFLNet [2] and Nyx-Net [7] on ten 24-hour trials against ProFuzzBench targets. We patch Nyx-Net specifically to emit stability information, and collect the same from AFLNet. We empirically observe that existing network fuzzers are prone to instability (see Figure 3).

3.3. Tracking Liveness During Dynamic Protocol Analysis (Challenge 3)

In both fuzzing and other dynamic protocol analysis approaches, techniques for testing or analyzing the behavior of network protocols face a fundamental challenge of determining when to stop awaiting a response from the endpoint before moving on to the next test. For remote systems, protocol-specific knowledge of the presence or cardinality of responses for a given request is used in tandem with timeouts in a best-effort approach. However, for systems that are both intended to be tested locally and possess many inputs (e.g. in the case of fuzzing or property-based testing), awaiting timeouts for messages that fail to elicit a response leads to an expensive slowdown in testing performance. This is not an uncommon occurrence—protocols may send unsolicited messages (e.g., MariaDB and similar database wire protocols) or optionally send no response at all for certain inputs (e.g., Open5GS). In addition to this, protocol implementations may omit error responses to malformed requests or expect additional input beyond what was received via fuzzing due to length fields within the protocol payloads, thereby leading to no response. We refer to this problem generally as that of *Liveness Tracking*, or the ability of a network-based testing framework to identify that a program has fully finished processing a given input.

AFLNet, SnapFuzz [3], and SGFuzz [5] all use protocol-aware message tracking over a remote socket to attempt to identify when a system is finished. Fuzztruction-Net [11] goes one step further than this, specifically leveraging an existing client implementation as an oracle for determining end of execution. Such an approach limits the domain of protocols to those that have one-to-one request and response message flows and those with servers that do not routinely omit sending certain responses; even within these requirements, coverage may be dropped due to early shutoff of a fuzzing round.

On the other hand, Nyx-Net, netfuzzlib [8], and preeny [6] track liveness by intercepting I/O calls on the target network socket; this strategy uses an LD_PRELOAD harness combined with bookkeeping for file descriptors and associated actions. This technique, however, presumes a single-threaded context; it assumes that a blocking poll on the input connection’s file descriptor means the system as a whole is finished execution. As this assumption can easily be violated when multiple threads are active, it is therefore ill-suited to multithreaded programs.

To wit, target program may employ one thread to handle communication while passing information to other threads for processing without an explicit response indicating such an action. If the target application continues executing after responding, the fuzzer would receive incomplete coverage information. To make matters worse, such a cutoff in execution may lead the fuzzer to completely miss crashes that come late in the packet’s handling.

Ultimately, both of these general approaches can suffer from either missed crashes and misattributed crashes during sequential test cases. While this has clear consequences during fuzzing, more nuanced testing approaches such as property-based testing [12] would additionally suffer from the proliferation of false classifications of security properties. This limitation has hampered the development of high-performance property-based testing techniques for network protocols; a framework offering precise liveness tracking could thus enable further development and adoption of property testing techniques in network protocols.

3.4. Dynamic Analysis of Multithreaded Targets (Challenge 4)

Prior work has identified deterministic concurrency as a key challenge for evaluating and testing systems security and correctness properties, but has been limited to only the issue of concurrency in isolation of other sources of nondeterminism [13]. Whereas these approaches aim to identify concurrency bugs by exploring different thread interleavings, applying coverage-guided fuzzing to concurrent applications remains an open problem due to the challenges of instability and accurate coverage feedback from interleaving. In regard to multiprocess applications, most prior work either manually removes `fork()` / `exec()` procedures from targets prior to fuzzing or otherwise limits the scope to single-process applications [14]. As such, no prior work

has been proposed for *deterministically* fuzzing concurrent applications.

4. Fizzle

We design FIZZLE to overcome the challenges presented in Section 3—namely, ensuring that the system under test emits deterministic and reproducible output for a given input, and providing precise liveness tracking to notify a fuzzer when the system has finished acting on its inputs. Beyond these, we consider portability to be a key feature to the utility of FIZZLE for future work in network dynamic analysis—it should operate without requiring modifications to source or binary and handle a wide variety of implementation details (e.g., programming language, async/event framework being used) in an agnostic manner.

4.1. Overview

To accomplish these goals, FIZZLE interposes information flows between the application and the OS and performs user space emulation of key nondeterministic OS features (see Figure 4). FIZZLE exposes input and feedback routines to integrate with arbitrary fuzzing platforms (e.g. AFL++, LibAFL, Nyx) and includes a plugin system for defining I/O endpoints (sockets, files) to be emulated or fuzzed. We implement FIZZLE in 40k lines of Rust code that compiles into a single shared library. We describe the technical implementation of these key features below.

4.2. System Library Interposition

While a number of system call interposition mechanisms exist—`ptrace`, `seccomp/BPF`, binary rewriting, or even kernel module-based interposition [15]—they suffer from drawbacks in either reliability or portability. Modern kernels incorporate `vDSO`, which allows certain frequently-used system calls to be exported to user space without context switching. These include `gettimeofday` for precise and low-latency time measurement, which introduces nondeterminism via clock time measurements. As such, interposition methods that rely on monitoring user-kernel context switches (which include `ptrace`, `seccomp/BPF` and kernel module-based methods) would be incapable of providing complete determinism for applications that use timing APIs. While binary rewriting techniques would be able to account for these `vDSO` routines, they rely on the assumption that `syscalls` and/or OS library call sites can be precisely identified statically, and that sufficient unused space exists in the binary to write in trampoline routines for interposition. Both of these are open challenges in binary analysis, so portability across CPU architectures, operating systems and even individual applications would be a concern.

To provide a deterministic emulated environment for applications (**Challenge 1**), FIZZLE interposes system library calls by shared library preloading. Most Unix systems offer such functionality via a special `LD_PRELOAD`

Subsystem	Sample of APIs Emulated by Fizzle
Sync.	<code>pthread_mutex_*</code> , <code>futex</code> , <code>sem_*</code> , <code>pthread_spin_*</code> , <code>pthread_condvar_*</code> , <code>pthread_barrier_*</code> , <code>eventfd</code>
Threading	<code>pthread_create</code> , <code>pthread_exit</code> , <code>pthread_wait</code> , <code>pthread_cancel</code> , <code>pthread_join</code>
Forking	<code>fork</code> , <code>atfork</code> , <code>atexit</code> <code>exit</code> , <code>_exit</code> , <code>wait</code> , <code>waitpid</code>
I/O	<code>socket</code> , <code>pipe</code> , <code>write</code> , <code>read</code> , <code>write</code> , <code>preadv</code> , <code>pwritev</code> , <code>sendmsg</code> , <code>recvmsg</code>
Signals	<code>pthread_sigmask</code> , <code>pthread_kill</code> , <code>sigwait</code> , <code>sigaction</code> , <code>sigsuspend</code> , <code>kill</code> , <code>pause</code> , <code>signalfd</code>
Filestreams	<code>fopen</code> , <code>fdopen</code> , <code>fread</code> , <code>fwrite</code> , <code>fflush</code> , <code>getc</code> , <code>putc</code> , <code>fpurge</code> , <code>fprintf</code> , <code>fscanf</code>
Resolving	<code>getaddrinfo</code> , <code>gethostbyname</code> , <code>getnameinfo</code> , <code>getservbyname</code> , <code>res_query</code> , <code>res_nquery</code> , <code>res_send</code>
Entropy	<code>srand</code> , <code>rand</code> , <code>getrandom</code> , <code>getentropy</code> , <code>/dev/urandom</code>
Timing	<code>time</code> , <code>gettimeofday</code> , <code>alarm</code> , <code>getitimer</code> , <code>setitimer</code>

TABLE 2. GENERAL CATEGORIES OF APIs THAT EITHER A) INTRODUCE ENTROPY INTO A PROCESS, OR B) REQUIRE EMULATION TO ENSURE DETERMINISTIC THREAD LIVENESS TRACKING.

or `DYLD_INSERT_LIBRARIES` environment variable that specifies a shared library to first search for symbols from prior to other libraries (i.e., `libc`). Using this technique, FIZZLE reimplements and interposes function signatures in `libc` that would otherwise invoke system calls. Programming languages regularly depend on `libc` to interface with system calls (with exceptions noted in Section 6.3), so such interposition is generally effective at capturing all system calls an application would normally make. Direct system calls are likewise captured in this manner, as applications commonly use the `syscall()` system library function rather than handwritten assembly. In many `*nix` systems, handwritten assembly system calls are either blocked explicitly as a security feature [16] or are considered an unstable ABI that changes between versions [17]–[19], so programming languages and applications must use `libc` exclusively to interface with the operating system.

A wide variety of system library calls must be interposed and faithfully emulated in order to ensure that application behavior is preserved while execution is rendered sequential and deterministic. Not every system library function must be interposed—those that do not invoke system calls (such as `memcpy`) and those that do not bring nondeterministic behavior into the process (such as `fsync`) require no interposition. We provide a small sample of the system library functions FIZZLE interposes and emulates in Table 2, and describe the specific mechanisms used to emulate functional subsystems of the operating system in the remainder of this section.

4.3. Deterministic Multithreading

Multithreading is commonly used in network applications to facilitate resource scaling across many concurrent connections. As such, a network fuzzing harness must account for thread control flows that may pause and resume at nondeterministic points relative to one another, as well as concurrent data access by threads. This represents a particular challenge for coverage-guided fuzzing, which re-

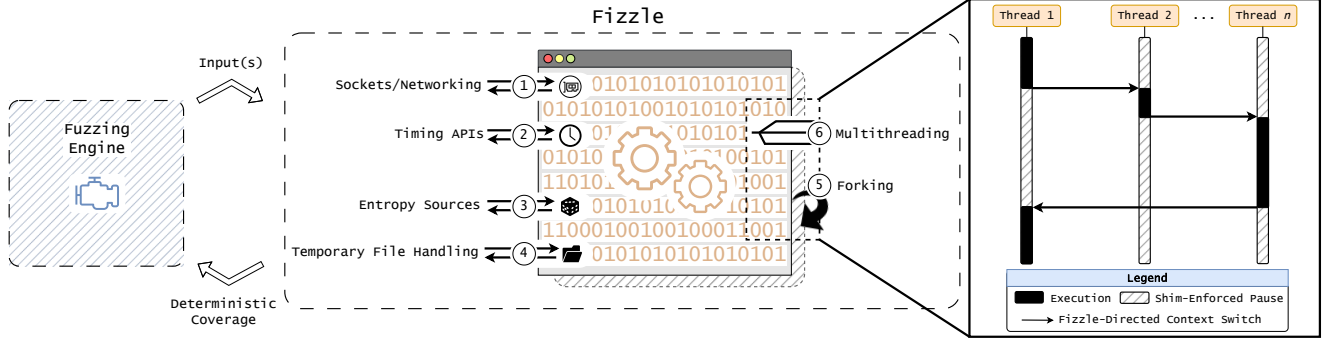


Figure 4. An overview of how FIZZLE operates. Target networking applications exhibit several dependencies and behaviors that pose challenges to current fuzzers, including (1) sockets/networking, (2) timing APIs, (3) entropy sources, (4) temporary file handling, (5) forking, and (6) multithreading. To address these challenges, FIZZLE forces applications to run as if they are single-threaded which enables deterministic coverage and reliable end detection.

lies on feedback from instrumented branch instructions to discover new/interesting execution paths. Nondeterministic thread execution order and interleaving produces inconsistent control flow feedback across consecutive fuzzing rounds; this translates to lower coverage stability, impaired fuzzing performance and difficulty in reproducing certain crashes.

To overcome this (**Challenge 4**), FIZZLE hooks system library functions related to thread/process handling (e.g., `pthread_create`, `fork`) and enforces strict sequential ordering of control flow. Enforcing thread ordering from user space is a nontrivial challenge; *nix operating systems do not generally supply APIs for fine-grained manipulation of thread scheduling. To overcome this limitation, FIZZLE associates a semaphore with each thread at the time the thread is created and maintains a scheduling queue that identifies what threads are ready to be executed. The primary thread is implicitly assigned control flow on process startup; when a new thread is spawned, the thread that spawned it adds itself to the scheduling queue and `wait()`s on its assigned semaphore. When a thread reaches a blocking operation—such as polling for unarrived socket input, suspending until a signal is received, or sleeping for a given duration—it delegates execution to the next ready thread by removing its semaphore from the scheduling queue and `post()`ing to it. Threads are scheduled in a simple First-In, First-Out (FIFO) queue to ensure they are ordered deterministically for a given fuzzing round; Appendix A shows the algorithmic steps taken for this process.

As described above, FIZZLE specifically ensures that at most one thread is executing *within the scope of the application* at a given time. An astute reader may note that during control flow delegation from one thread to another, the active thread must necessarily `post()` another thread’s semaphore prior to `wait()`ing on its own, thus creating a hazard zone where both it and the awakened thread may be live. FIZZLE handles this race condition by (a) requiring that no shared internal state is accessed between a `post()` and corresponding `wait()`, and (b) disabling coverage feedback from FIZZLE routines. By so doing, FIZZLE elim-

inates nondeterministic behavior in multithreaded coverage feedback—only one thread at most runs actual application code and therefore emits coverage at a given time, and scheduling is always ordered identically each time a given fuzzing input is run.

4.4. Shared State

While rudimentary tracking of live threads can be accomplished using only semaphores and a global scheduler queue, such an approach would fail to account for resources that are dynamically unblocked/blocked through the lifetime of a program. Multithreaded programs are regularly designed to synchronize and pass information between threads; as such, tracked threads must be re-added to the scheduler as the resources they block on become available. Thus, FIZZLE additionally interposes and emulates system library functions that conditionally block and/or unblock threads to internally book keep their state and perform scheduling where appropriate.

Some inter-thread communication means are relatively straightforward to model: semaphores, mutexes and condition variables are generally implemented by FIZZLE using simple counters and queues of thread/process IDs to deterministically decide the next awaiting worker to lock the resource for. Others, such as I/O events in file descriptors, necessitate more sophisticated tracking—one resource may have multiple `dup()`ed file descriptors that are awaiting an input, and simultaneously one thread may poll on multiple descriptors pointing to a variety of underlying I/O types. Once events are detected, FIZZLE is then tasked with correctly simulating the passing of data through pipes, sockets, eventfds or other mediums via system library calls that may include vectored buffers, per-message destination addresses, and/or ancillary data. Communications may additionally be buffered via `FILE*` streams, thereby necessitating the interposition of functions related to their operation as well.

In certain cases, FIZZLE must pre-empt a task to be performed prior to running threads already scheduled actively; such is the case for signal handling. In order to faithfully

model the potential re-entrant nature of signals, FIZZLE immediately suspends and pushes the current running thread to the front of the execution queue when it calls an API that would emit a signal. It then records that a signal is being passed in internal state and delegates execution to the receiving thread or process. When that thread/process awakens, it checks for an incoming signal and consumes it, immediately executing the handler where applicable or terminating the thread/process. Both per-thread and per-process signals are emulated by FIZZLE, as well as signal handler installation, blocked/pending signals, and automatic signal transmission when certain actions are carried out (i.e., SIGCHILD, SIGPIPE, SIGIO).

4.5. Fuzzing Lifecycle/Liveness Tracking

As discussed in Section 3.3, accurate liveness tracking is a prerequisite to efficiently running both deterministic fuzz testing and further property-based testing techniques for network systems (**Challenge 3**). Rather than relying on some loosely associated heuristic for tracking liveness (as prior network fuzzers have opted for), FIZZLE directly tracks state for each thread and/or forked process in an application.

4.6. Root of Entropy

Beyond thread nondeterminism and liveness tracking, FIZZLE interposes system library functions that would inherently provide differing outputs across sequential runs. These range from explicit sources of pseudorandom bytes such as `getrandom`, `/dev/random`, etc., to clock/timing APIs and timestamp metadata attached to files, to process/thread ID values. The interposed functions then return plausibly valid data—for instance, clock/timing APIs return monotonically increasing intervals on successive calls.

A key observation is that FIZZLE aims to be deterministic and emit identical coverage feedback *for the given fuzzing input it receives*. Execution runs that begin with a different input value are not constrained to run identically; as such, FIZZLE can take several liberties to introduce pseudorandom behavior into the environment the program runs in based on its received fuzzing input. To wit, FIZZLE doesn't leave the environment of a program constant—it can still provide the application an environment that behaves randomly (or even erratically) in places a normal environment would. The key difference is that FIZZLE derives every random interaction it produces from the current fuzzing input, thereby enabling a crash from a given input to be replayed and debugged with precision provided the crashing input is saved. This property overcomes the long-standing challenge of crashing seeds failing to reproduce following a fuzzing campaign.

4.7. Pluggable Network Dependencies and Configuration

Up to this point, we have identified and accounted for nondeterminism stemming from environmental inputs and

cross-thread/process interactions. While for some applications this may be sufficient, many network systems operate under the additional assumption of access to certain external network dependencies (e.g., DNS recursive resolvers). This would involve communication to/from said remote dependency and polling that awaits its responses (**Challenge 2**); allowing such would negate the ability of FIZZLE to track program liveness. To overcome this issue, FIZZLE defines a plugin architecture that enables simulated clients and servers of protocols to communicate with the target application. Plugins are written as separate Rust modules and added to FIZZLE at build time, and they may leverage and re-use existing client/server crates for simulating protocols.

FIZZLE exposes a simple configuration file that enables one or more I/O endpoints to be passed fuzzing input. We show an example instance of this configuration for an FTP server in Listing 1. Unlike prior work, FIZZLE is capable of seamlessly passing fuzzing input via *arbitrarily many input channels* without requiring any modifications to code; as little as two lines of configuration are sufficient to begin fuzzing a client or server instance.

```
# Reset after each fuzzing round
tmpfolders = [ "/home/user/ftpshare" ]

# I/O endpoint to be passed fuzzing input
[io]
"tcp-client:127.0.0.1:21" = "fuzz"

# I/O endpoint to be mocked
[io."udp-server:127.0.0.1:53"]
method = "plugin"
when = "onstartup"
module = "fizzle-plugin-dns"
plugin = "DnsMockServer"
```

Listing 1. A sample configuration for fuzzing an FTP server with FIZZLE.

5. Evaluation

In this section, we describe our experimental setup, detail our selected datasets and evaluate the performance of FIZZLE in comparison with two existing fuzzing frameworks, AFLNet, and Nyx-Net.

5.1. Experimental Setup

Advances in network fuzzing research over the past 5 years have generally been evaluated holistically—the fuzzing engine and executor are evaluated together with no ablation study on the effect of each. Papers such as AFLNet and Nyx-Net offer improvements to mutation strategies and state feedback techniques in addition to the manner in which input is passed in and environment reset. FIZZLE, however, offers a framework agnostic to any one backend fuzzing engine; any manner of input mutation or scheduling algorithms may be applied to it, as it focuses exclusively on the challenges of passing in network input and deterministically receiving resulting coverage. As such, evaluating it against Nyx-Net and AFLNet directly would introduce confounding variables due to potentially significant disparities in the efficacy of

techniques used by the backend fuzzing engines of these prior works.

To resolve this, we perform a full port and adaptation of Nyx-Net’s `ld_preload_fuzz.so` executor and AFLNet’s socket-based executor into the LibAFL fuzzing framework. We preserve interactions and code to the fullest extent possible while doing so, and resolve any observed regressions in performance during the adoption before evaluating. We additionally write a LibAFL executor that integrates with FIZZLE for packet inputs. We then configure identical back-end packet mutation operations, power scheduling, and coverage feedback handling for each executor to ensure a level evaluation for the otherwise disparate fuzzing executors.

Fuzzing Methodology. For each $\langle \text{Server}, \text{Executor} \rangle$ tuple, we ran ten 24-hour fuzzing campaigns to help ensure findings are repeatable and significant, per recommendations from Klees et al [20]. We use the same seeds for each cross-evaluated fuzzing campaign and operate individual fuzzers in isolation from each other. We use the fundamental approaches taken by AFLNet and Nyx-Net (as well as subsequent network fuzzing research that make use of the same input techniques) as a baseline with which to compare Fizzle against, in line with best practices outlined by Schloegel et al [21].

Hardware Setup. We evaluate on a Supermicro 1U WIO A+ SuperServer (AS -1014S-WTRT) server running with 64 CPU cores, 128 threads, 256 GB of RAM and an SSD capable of 98,000 iOPS read and 88,000 iOPS write. Each fuzzing campaign is pinned to a single, distinct core from other consecutive campaigns via `cgroups` within an isolated docker container.

5.2. Datasets

For robust evaluation of FIZZLE, we select two datasets that encompass a wide variety of network server behaviors and types.

ProFuzzBench (D_{PFB}). We first consider the ProFuzzBench [22] dataset, which includes 13 fuzzing targets that have been used in prior network fuzzer comparisons. These benchmarks encompass ten different network protocols (RTSP, SMTP, FTP, SSH, TLS, DTLS, DNS, SIP, DAAP, and DICOM). We exclude DAAP and its associated implementation Forked-DAAPD, as this protocol requires external dependencies of the `systemd-DBUS` and the `avahi-daemon`, and requires passing through underlying DBUS/Avahi daemon access to docker containers, which is non-trivial for evaluation involving simultaneous Dockerfiles.

ProFuzzBench (D_{PFB}^*). Next, we select four of these targets, (Kamailio, Exim, TinyDTLS and OpenSSH), for further coverage evaluation. Several of the servers in ProFuzzBench are different implementations of the same underlying protocol; beyond this, our analysis of multithreaded implementations (detailed below) includes DNS and FTP

servers. Therefore, we focus more comprehensive coverage tests on this smaller, deduplicated subset of ProFuzzBench selected for their unique properties. Kamailio was chosen over other HTTP-like protocols, because it produces the largest fuzzing queues of all targets. Exim has strict request pipelining requirements. Finally, TinyDTLS offers exploration of a different transport-layer protocol (UDP) compared to the other ProFuzzBench targets.

Further, we observe that ProFuzzBench test suite features several limitations that render a comprehensive study incomplete. First, all ProFuzzBench targets require manual modifications to source code to: remove forking from execution, artificially reduce instability by swapping out entropy sources with fixed values, and remove similar sources of environment instability. Moreover, ProFuzzBench lacks a single multithreaded implementation of a network server, instead, all use `forkserver` models that are subsequently disabled to effectively replicate sequential blocking servers. Additionally, all asynchronous/event handling techniques used in post-C10K server architectures are absent from any of the targets. Finally, protocols in ProFuzzBench are nearly all textual rather than binary, have clear packet delineation, and involve one-to-one request responses. This precludes evaluation against protocols that may violate these implicit assumptions.

Multithreaded Server Protocols (D_{MTSP}). To provide FIZZLE with a more representative dataset, we additionally evaluate against a suite of multithreaded network server protocols, Unbound DNS, FileZilla FTP server, MariaDB, Redis, and Open5GS. The former two use the same protocol as in previous ProFuzzBench targets but employ explicitly multithreaded architectures. The latter targets are both multithreaded and violate the above assumptions.

5.3. Experimental Methodology

Disparate Stability Detection Mechanisms. During the process of adapting each fuzzer, we made the curious observation that the stability metrics used for Nyx-Net, AFLNet, and LibAFL are all calibrated by different means. AFLNet defaults to re-running each input being added to the queue eight times; if any difference in coverage is identified between those runs, it runs an additional four times, marks all identified unstable edges as permanently ignored, and then schedules up to two additional twelve-run calibration stages for that input. To wit, each unstable input being added to the fuzzing queue will be replayed *up to thirty-six times* during the process of fuzzing.

LibAFL, on the other hand, takes a much more lean approach, defaulting to four repeat trials of each input entering the queue. Each run instability is detected, it increases the total number of runs by two up to a maximum of twelve reruns for a given calibration. Nyx-Net doesn’t report stability as one of its metrics; although it performs a calibration stage of 10 reruns for each new seed in the queue, it does so only to reject inputs that do not consistently produce new coverage

feedback. Unfortunately, because unstable edges are not tracked across individual executions, the consequence of this design is that unstable edges that are commonly reached will repeatedly trigger calibration. Eventually, inputs with unstable edges are added to the queue anyways, provided they run a sufficient number of times to have at least one consistently-reached edge, thereby negating the benefit of the calibration stage. For our experiments, we adopt an approach similar to that used by AFL++ (and correspondingly AFLNet), as it offers the best chance at detecting and mitigating instability during calibration.

Evaluating Coverage Outcomes across Executors. When evaluating fuzzing executors, one encounters the challenge of how to fairly compare the coverage feedback emitted by disparate executor techniques. Both the input mechanism and the manner in which the environment is reset may unduly inflate or shrink coverage values and correspondingly assign more coverage to one executor technique than another. While using a single standard executor to evaluate the input queues of others may seem to account for some of these issues, it runs into the challenge of fuzzers favoring coverage that their executor specifically favors exploration of.

In evaluating FIZZLE, both Nyx-Net and AFLNet executors record coverage at differing times with respect to one another and FIZZLE. The Nyx-Net executor records coverage from program start up to the point all inputs have been read and the server prompts for an additional read. At that point, Nyx-Net immediately cuts off execution. Meanwhile, AFLNet explicitly closes a socket and sends a termination signal to the process without any delay after inputs are exhausted, therefore collecting additional coverage reached during connection termination routines.

FIZZLE takes the approach of recording precisely the coverage that may be reasonably influenced by the fuzzing input. Namely, it begins recording after all tasks for initializing the server have been completed and only polling for incoming connections remains. It records coverage while the underlying transport connection is established and throughout the process of sending/receiving inputs. When the transport connection has been shut down, coverage is recorded up to the point when all actions taken by the server in association with that shutdown are complete. By so doing, FIZZLE ensures that use-after free, double-free or similar vulnerabilities latent in the shutdown of a network connection are properly attributed to the correct fuzzing round.

The above observations mean that coverage is recorded differently in the following three contexts: 1) on server startup, 2) at transport connection close, and 3) on server shutdown/termination. To account for these differences, we opt to measure the delta in coverage observed when initial input seeds are run vs subsequent inputs to the queue. We consider this to be fair, as improvements in coverage during the connection life cycle are desirable for fuzzing; any coverage improvements from the baseline missed by Nyx-Net’s approach limit its efficacy and so remain included. On the other hand, this approach also includes changes and coverage that could artificially inflate AFLNet’s score due to

differences in termination routines—the ability to terminate a network server arbitrarily is not generally considered to be within the capabilities of an adversary. While this is a limitation of this metric, our results will show that FIZZLE sufficiently outperforms AFLNet regardless; our findings therefore represent a lower bound on FIZZLE’s efficacy vs. AFLNet.

5.4. Overhead of Stability Mitigation for Existing Fuzzers

To evaluate **RQ1** (*Do existing mitigations to instability (e.g., calibration stages) impose a significant overhead on fuzzing performance?*), we aggregate execution speeds over 24 hours, total seed inputs in scheduler queues, and unstable edge proportions at the time of find for each queue entry. We consider $\text{Cal}_{\text{Penalty}}$, or *Calibration Penalty*—the proportion of fuzzing executions that are spent on re-running calibration tests, as such:

$$\text{Cal}_{\text{Penalty}} \geq \frac{\sum_{i=1}^n \text{Cal}_i}{DS}$$

where

$$8 \leq \text{Cal}_i \leq 32$$

In the above equation, n is the number of queued items by the end of the fuzzing campaign, Cal_i is the number of calibration re-runs performed for the i^{th} seed input, D is the duration in seconds of the fuzzing campaign, and S is the speed in executions per second of the campaign.

We find that calibration imposes a nontrivial overhead to fuzzing, more especially so in the very cases where it is needed most—applications that take longer to run (due to size/complexity) also have more branches to be reached, leading to quadratic growth in $\text{Cal}_{\text{Penalty}}$. In MariaDB, we find re-running inputs to calculate stability accounted for **more than 40% of all executions** for both Nyx-Net preload and AFLNet campaigns over 24 hours. An approach that provides a fully deterministic environment and eliminates the need for *any* stability calibration therefore offers a critical performance improvement when fuzz-testing such systems.

As a final note, the equation listed in this section for $\text{Cal}_{\text{Penalty}}$ represents a *lower bound*, as calibration runs are applied on inputs that do not eventually make it into the queue. This is the case when an input initially shows new coverage, but all new coverage edges it has are found to be unstable during the calibration stage.

5.5. Fuzzing Exploration

To evaluate **RQ2** (*Does environment emulation and interposition allow for more efficient exploration of the target program?*), we compare FIZZLE with existing fuzzers by performing experiments on $\mathbf{D}_{\text{PFB}}^*$ and $\mathbf{D}_{\text{MTSP}}$ and observing coverage results. We highlight results from these in Figure 5 and Figure 6, respectively. Overall, we observe

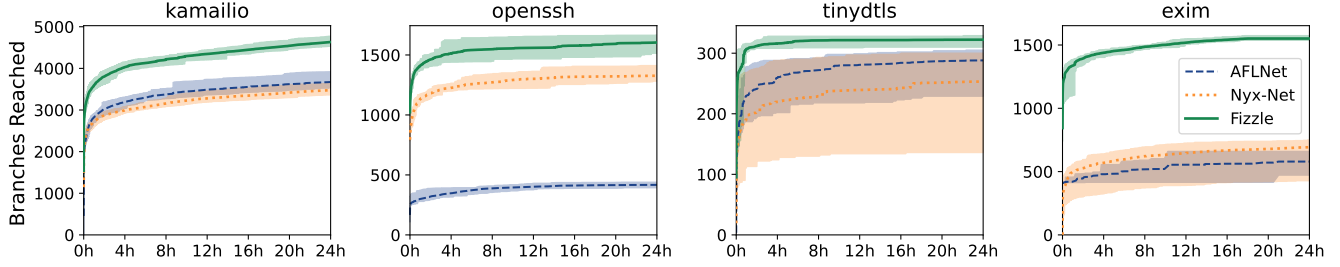


Figure 5. Coverage for LibAFL runners for AFLNet, Nyx-Net (preload) and FIZZLE frameworks on D_{PFB}^* over 24 hours. Error bars indicate 90% confidence interval.

that FIZZLE achieves better coverage than both existing fuzzers. Unlike previous work, FIZZLE is not forced to ignore unstable inputs to its queue and can therefore efficiently explore branches of a target program. Furthermore, our system maintains a higher execution rate throughout its runtime. We explore details of each benchmark test below. It should be noted that in many cases, our system reaches higher coverage in 1-4 hours than other fuzzers can achieve at 24 hours, demonstrating that factors other than merely improved execution speed (e.g., zero feedback loss from instability) contribute to FIZZLE’s effectiveness.

Kamailio, a SIP server for telecommunications, relies on TCP for message exchanges. As a fuzzing target, Kamailio exhibits output queues containing thousands of seeds at the end of a 24-hour campaign. At this scale, the cost of instability is magnified. Figure 5 exhibits the implications of this loss, with FIZZLE able to find over 1000 additional branches when compared to Nyx-Net and AFLNet. When analyzing the output queue size from logs generated by each fuzzer, we see further evidence of the efficiency of our system (≈ 8000) compared to Nyx-Net (≈ 4500) and AFLNet (≈ 2000).

Our second benchmark is OpenSSH, a popular open-source SSH server. Results shown in Figure 5 reveal the limited efficacy of AFLNet in this setting, namely in branches reached. This result can be attributed to the small timeout value that AFLNet uses for targets, ultimately causing the fuzzer to terminate the server before the server can process the packets sent to it and generate a response.

TinyDTLS is a library implementing Datagram Transport Layer Security (DTLS) for clients and servers, though we target only the UDP server application for these experiments. Surveying Figure 5, we see a large spread in coverage values between runs of Nyx-Net and AFLNet, a symptom of instability during fuzzing campaigns. In this setting, implementations draw from entropy for key exchanges, which introduces instability into the system. Since FIZZLE can derive entropy calls from the input and features an internal random number generator, however, it can provide high-quality yet deterministic randomness in this context. Such a property further avoids the drawbacks of introduced instability during a fuzzing campaign.

For Exim, a program implementing SMTP, we observe that our approach doubles the number of new branches

reached compared to existing fuzzers. Through these experiments, we discovered that Exim will immediately reject and close any connection that (a) attempts to send data before it has sent a banner message to the connection, or (b) pipelines the request (e.g., sends a second request before the response to the first one has been received). When performing this experiment with Nyx-Net, we had to edit the Nyx-Net preload fuzzer to prevent the former issue; the authors of Nyx-Net handle this problem by taking a snapshot after Exim finished sending its banner. Despite these precautions, the preload shim still sends requests in a pipelined manner, which inevitably leads to connection rejection from Exim after a short period. Meanwhile, the AFLNet fuzzer suffers because its fixed ‘send()’/‘recv()’ timeouts behave poorly with the variably-timed responses from Exim. Since FIZZLE can monitor the lifetime of a target, these issues are not present in its fuzzing campaign.

These experiments on single-threaded targets ultimately show that different targets exhibit different challenges to a fuzzer (e.g., Nyx-Net achieves better coverage for OpenSSH than AFLNet but not for TinyDTLS). Despite these different challenges, FIZZLE offers the best coverage on all four targets, highlighting its robustness. Finally, we note here that some previous fuzzing techniques (e.g., Nyx-Net) do not explore coverage once a connection is set to be closed (i.e., the fuzzer ends execution whenever the target looks to read in additional input) while others (e.g., AFLNet) stops executing after the program has ended and therefore report inflated coverage values (i.e., the fuzzer added entries to the queue during shutdown that do not provide value). Using a full view of the target program, our system can cover the entire lifetime of a program without missing branches or inflating coverage values.

For D_{MTSP} , we focus analysis on two multithreaded targets: Unbound and Redis. For Unbound, the Linux kernel permits the binding of multiple sockets to the same listening address. The program binds to eight different sockets on different threads, all bound to the same address. Nyx-Net relies on observing that a target program is ready to receive a packet to indicate that the program is done processing input; however, with eight available threads, the fuzzer concludes that Unbound has finished processing input immediately because threads not handling the packet trigger read polling on the input file descriptor. As a result, packets passed in

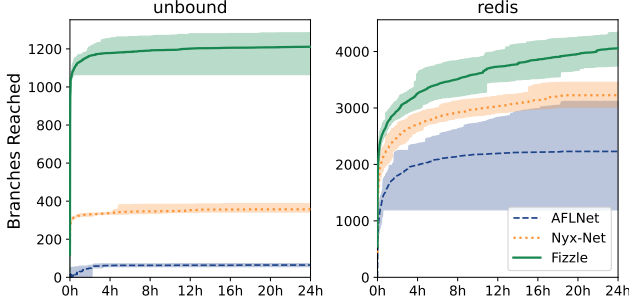


Figure 6. Coverage for LibAFL runners for AFLNet, Nyx-Net (preload) and FIZZLE frameworks on Redis/Unbound over 24 hours. Error bars indicate 90% confidence interval.

do not have time to be processed before the fuzzer moves onto its next round. Meanwhile, AFLNet uses a response as an indicator that Unbound has completed processing a packet (with associated static timeout values) and therefore misses responses from Unbound when that timeout value is hit. Given that Unbound exhibits variably-high startup time, this behavior is unsurprising. Since FIZZLE operates without these assumptions, it reaches at least three times as many new branches compared to either fuzzer.

Unlike Unbound, Redis does not implement behavior that would cause its input mechanism to exhibit instability. Namely, Redis supports pipelined requests, binds to one socket, and finishes processing one request before receiving another. These properties make it an ideal candidate for both AFLNet and Nyx-Net, as the assumptions made by both fuzzers to determine when a target is finished processing input are valid in this context. Despite this, FIZZLE reaches more branches than either fuzzer.

5.6. Deterministic Replication of Fuzzing Campaigns

We next explore **RQ3** (*To what degree can the environment and execution of networked, multithreaded applications be made deterministic without fundamentally modifying their code?*) by applying identical seeds to FIZZLE and comparing outputs from concurrent runs.

Along with demonstrative improvements in speed, FIZZLE offers validation properties that prior work cannot. Namely, FIZZLE produces seeds leading to crashes that are bit-for-bit reproducible in *both* content and number of executions. This property ensures that, agnostic of the host machine, a given seed will result in a crash exactly at the expected number of executions—though the exact clock time in which the vulnerability was found may vary slightly. This property reaches to the limits of reproducibility for dynamic analysis and execution in that an improvement here (i.e., guaranteeing identical execution clock time) is infeasible given fundamental fluctuations in both host operating system and hardware.

Although identical seeds should intuitively produce the same behavior, this is not the case for existing fuzzing tech-

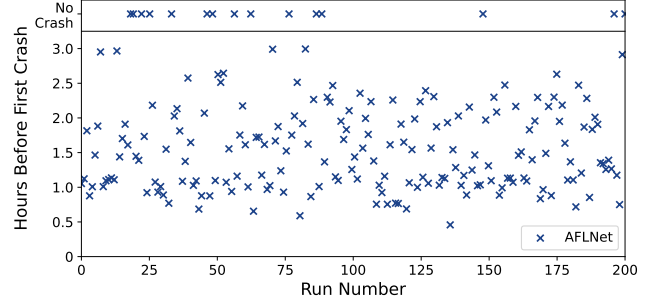


Figure 7. The time needed to cause the program bftpd to crash in consecutive runs using AFLNet with *identical* seed. We observe a messy distribution of data points indicative of high entropy within the population, ultimately showing that even identical seeds will produce unreliable results in existing fuzzers.

niques. To highlight this behavior, we apply identical seeds to the AFLNet fuzzer when testing on the target BFTPD and observe varying results across runs, sometimes without ever producing a crash (see Figure 7). When performing a similar test for Nyx-Net, we observed different execution runs despite a period of 100% stability and identical seeds passed to the random number generator of the backend fuzzer. This behavior suggests that the feature is not implemented correctly; therefore, we omit Nyx-Net from this analysis. When performing 300 similar runs using FIZZLE, however, we observed identical queues, execution numbers, and crash locations.

5.7. Fuzzer Execution Speed

Next, we evaluate **RQ4** (*How does program environment interposition/emulation impact execution speed for fuzzing?*) by comparing FIZZLE’s performance with that of AFLNet and Nyx-Net. We execute **D_{PFB}** for 24 hours and measure the average executions per second across ten runs of each target program. As Figure 8 shows, FIZZLE doubles execution speed in most cases and performs 20x faster than other network fuzzing executors in specific scenarios.

We observe that AFLNet exhibits the worst performance among evaluated frameworks. This is most likely due to the techniques use of timeouts on start-up, and between every packet sent and received. These timeouts upper bound absolute performance as they are required waiting periods before more inputs can be sent, which scales linearly with the number of packets transmitted. This greatly penalizes performance in stateful fuzzing where a significant number of inputs are sent and received.

In contrast, Nyx-Net interposes all `read()`, `recv()` and similar system library calls to look for the first chance of sending data through a connection to the application. Unlike AFLNet, there are no intentional slowdowns or timeouts that would hinder performance. As such, it provides a good comparison for FIZZLE, which likewise does not include slowdowns or timeouts but additionally necessitates bookkeeping of various state information (described in Section 4). If these

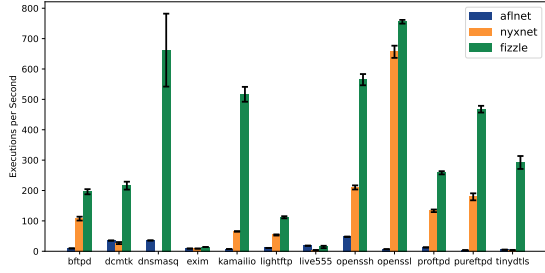


Figure 8. Execution speed of LibAFL runners for AFLNet, Nyx-Net (preload) and FIZZLE frameworks on $\mathbf{DPF}$ over 24 hours. Error bars indicate 90% confidence interval. Note that Nyx-Net is not measured for dnsmasq due to unimplemented `recvmsg()` functionality in their artifact.

operations introduce an overhead on performance, we would expect observe a decrease in execution speed relative to Nyx-Net. Instead, such emulation appears to provide significant performance benefits to fuzzing speed for FIZZLE.

As a second frame of reference, we monitor resource utilization on cores running each of the three fuzzing executors. For cores pinned to FIZZLE, utilization was consistently pegged at 100% throughout experimentation. In contrast, Nyx-Net campaigns exhibited varying degrees of drops in utilization and spikes in the proportion of CPU usage consumed by kernel space operations; AFLNet campaigns exhibited consistently low CPU utilization due to timeout usage. This is most likely explained by frequent kernel context switching during Nyx-Net experimentation, necessitated by the framework’s use of socket connections, mutexes and similar kernel structures. The effect became more pronounced as many campaigns were simultaneously run on disparate CPU cores, indicating that preload-based fuzzing may suffer linear scaling issues for larger fuzzing campaigns. Meanwhile, we observed that FIZZLE effectively fuzzed upwards of 120 simultaneous fuzzing campaigns on a 128-core machine without any observable degradation in CPU utilization. In effect, FIZZLE *transforms* I/O- and synchronization-intensive operations into zero-context-switch emulation functions that more effectively utilize per-campaign CPU allocations.

Persistent Channel Reset. Lastly, FIZZLE presents a special configuration we name as Persistent Channel Reset (PCR) mode. This option allows FIZZLE to reset only the transport connection, reusing the same copy of an application across multiple consecutive fuzzing rounds. This introduces some instability within the application’s memory, which is not reset as would be typical operation, as a trade-off for dramatic increases in execution speed. This results in over a 10x *additional* increase in performance for our multithreaded network server test suite as seen in Figure 9, when compared with standard FIZZLE. The number of persistent rounds is configurable at runtime; in practice, we note that the optimal round count varies across across targets but can be identified via trial and error. While PCR mode is unsuitable for fully deterministic/reproducible fuzzing campaigns, it

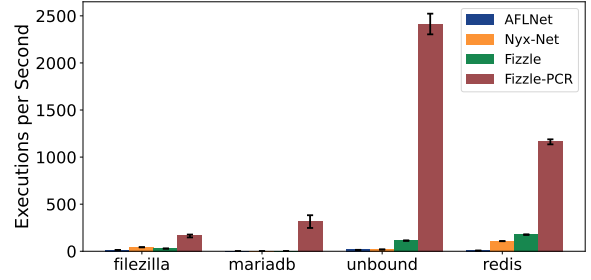


Figure 9. Execution speed of LibAFL runners for AFLNet, Nyx-Net (preload), FIZZLE, and FIZZLE-PCR frameworks on $\mathbf{DMTSP}$ over 24 hours. Error bars indicate 90% confidence interval.

offers a much more appealing executor in scenarios where execution time is constrained and the goal is to quickly test for the existence of bugs, such as in Continuous Integration runners.

5.8. Case Study: Open5GS Cellular Core

Finally, we investigate **RQ5** (*Is Fizzle capable of finding new vulnerabilities in target protocols?*) by running fuzzing campaigns using FIZZLE. As noted in prior work, cellular 5G Core implementations violate many of the implicit assumptions existing network fuzzers rely upon. Communication protocols between the radio network and 5G may involve several responses or no response at all for a given request; require complete deserialization to delineate sequential packet inputs; have external network dependencies that are fundamental to their operation; and, in the case of Open5GS, be architected in multithreaded worker architecture. As such, neither Nyx-Net nor AFLNet executors are suitable candidates for fuzzing.

RANSacked [23] worked around this challenge by manually annotating points at which the server had completed processing packets and communicating this back to the fuzzer via an out-of-band channel. We adapt the backend fuzzing engine used in this work to run with FIZZLE and fuzz the same Open5GS core without requiring any such annotation or modification to its source code. In addition to finding the same crashes found by RANSacked at more than double the execution rate, FIZZLE discovered three additional vulnerabilities that were not previously uncovered by RANSacked. Of these vulnerabilities, two were the result of Transport-layer parameters in SCTP that FIZZLE could trivially mutate and pass semantically invalid values for to the MME and AMF components of Open5GS. Such vulnerabilities could have not been found by RANSacked, and are not reachable by either AFLNet or Nyx-Net more generally. As such, FIZZLE demonstrates it is capable of finding additional categories of vulnerabilities not discoverable by existing fuzzing works.

6. Discussion

6.1. Responsible Disclosure

For new vulnerabilities discovered by FIZZLE, we notified project maintainers privately. We identified root causes and offered potential patches for each bug, and followed best practices in disclosing vulnerabilities publicly once fixes were applied and available within a reasonable time window.

6.2. Applications Beyond Fuzz Testing

In this paper, we have explored the specific use of FIZZLE for deterministically fuzzing networked systems. FIZZLE is equally applicable to any user space system prone to degraded stability, such as decoders that make use of multi-threading or otherwise isolate applications that call time/randomness APIs. Beyond this, FIZZLE may enable a range of additional techniques for program dynamic analysis:

- **Fault injection** - By default, FIZZLE emulates a reliable network, file system and general environment free of spurious system call interrupts. Since FIZZLE interposes and controls output from such APIs, future work may consider adding various forms of network/disk fault injection to test systems where high availability and recovery are critical.
- **Thread ordering issues** - FIZZLE keeps track of the status of all blocking concurrency primitives, and its sequential handling of multi-threading is compatible with the general book-keeping approach used by ThreadSanitizer to identify race conditions. Moreover, FIZZLE can control the order or timing at which signal handlers are called, potentially enabling dynamic detection of the distinct category of `async-signal-unsafe` exploits.
- **Dynamic taint analysis of randomness** - FIZZLE can be adapted to provide chosen values or patterns for randomness APIs of varying levels of security, ranging from `rand` to `getentropy` to `/dev/urandom`.
- **Timing-related vulnerabilities** - Applications frequently rely on the assumption of monotonically-increasing time values from APIs that provide no such guarantee (or may even be influenced by remote Network Time Protocol calls). FIZZLE can readily be adapted to selectively return spurious timing values from specified clock sources.
- **Unit/Integration Testing for I/O** - Significant technical effort has gone into developing techniques for mocking, injecting or otherwise removing code components that interact with external nondeterministic environments, particularly for file- and network-based operations. FIZZLE may be considered in such contexts as an alternative application-level approach for handling environment dependencies.

6.3. Limitations

Raw System Calls. While most programming languages interface with the operating system via dynamically-linked

`libc` routines, Go uses raw system calls that are left uncaptured by FIZZLE. However, Go does use `libc` when compiled to different platforms without stable/accessible system call ABIs (e.g., macOS, OpenBSD, IllumOS); such cross-compilation can be a potential workaround for this case. Additionally, FIZZLE supports the `syscall()` `libc` function, which is commonly used in place of raw assembly system calls for applications that aim for portability. While interposition techniques such as `ptrace` offer paths to overcoming this obstacle, the notable performance benefits of FIZZLE would be negated due to the added overhead of frequent context switches, and raw assembly calls to `vDSO` would remain a gap in completeness.

Atomic Primitives. FIZZLE enforces deterministic, sequential execution of multithreaded applications. As such, systems that use hand-crafted busy-wait primitives (e.g. looping on an atomic load) instead of the commonly-used `pthread_spinlock` APIs will hang. Busy-wait loops that `yield` or include any kind of kernel-waiting mechanism are unaffected by this limitation.

Stateful and Complex Input Generation. Network protocols routinely include stateful components (such as message counters) and produce complexly-encoded packets with strict requirements in early decoding (such as encryption and checksums). Such inputs may be difficult to both correctly model and meaningfully fuzz, and FIZZLE defers this challenge to plugin implementation. Recent work such as Fuzztruction-Net takes an entirely different approach to input generation, instead employing fault injection on a reference client implementation to generate packets that are semantically valid to a much greater depth without requiring manually implementation of protocol grammars. While FIZZLE in its current form is limited to protocol- and structure-aware fuzzing approaches, it may be adapted in future work to permit simultaneous execution of both client and server within one deterministic environment, thereby enabling FIZZLE and Fuzztruction-Net to be used in a complementary fashion.

Completeness. FIZZLE aims to accurately emulate system library calls; our experiments have shown this to be effective for a variety of programs, I/O methods, and IPC channels. Nevertheless, this result does not imply that our approach guarantees full and faithful emulation of every `*nix` environment or system library call. FIZZLE experimentally demonstrates the potential benefits of deterministic userspace emulation on an initial set of fuzzing targets, and aims to be a framework to be built upon for ensuring determinism in a variety of dynamic analysis techniques. Consequently, FIZZLE is not a one-off contribution—continued refinement and open-source collaboration will lead to more complete and faithful system library interposition.

7. Related Work

While the term *fuzz* hearkens back to a 1990 study testing UNIX utilities [24], many mark the release of American Fuzzy Lop (AFL) as the start of widespread research and

adoption of coverage-guided fuzzing techniques. Such research has brought advances in fuzzing mutation strategies [25]–[28], complex grammar exploration [29], [30], coverage feedback techniques [31], [32], and best practices for benchmarking and reproducing fuzzing results [33], [34]. LIBAFL [9] synthesizes such research advances into a modular framework, making fuzzing advancements more accessible. FIZZLE likewise aims to bring together disparate approaches to stateful network protocol fuzzing under a performant, deterministic framework.

Network-Enabled Fuzzing. While blackbox generative fuzzing has widely been applied to network applications for over two decades [35]–[38], coverage-guided network protocol fuzzing has only recently materialized. AFLNet [2] emerged as an extensible fuzzer that employs coverage feedback to prioritize mutation selection while inferring state from server responses; later work improved on its performance [3], [39] and employed further compile-time instrumentation to infer state [40]. Nyx-Net [7] devised a novel incremental snapshotting approach to network fuzzing dramatically reduce program reload time and improve depth of exploration, while SGFuzz [5] focused on sophisticated mutation and state tree construction techniques to improve coverage. Aside from coverage-guided techniques, recent work has likewise advanced remote blackbox fuzzing and inference of unknown protocols [41], [42]. The above works all rely on either rudimentary `LD_PRELOAD` shims or over-the-network inputs with protocol-aware request/response tracking to delineate input coverage, both of which are subject to the limitations described in Section 3.2.

Similarly, utilities now enable fuzzing of network protocols directly using traditional greybox fuzzers [4], [4], [6], or else facilitate remote blackbox fuzzing of network services [43], [44]. Our prior work [23] attempted to overcome the limitations of network fuzzing frameworks by manually instrumenting notification channels indicating when the application has finished processing input. FIZZLE expands on these efforts to provide an extensible platform for network fuzzing that guarantees both fuzzing round consistency and deterministic coverage feedback.

Deterministic Simulation Testing. To counter the issue of being unable to reproduce bugs in deeply distributed/concurrent systems, various Deterministic Simulation Testing (DST) frameworks have been designed in recent years. For the most part, DST frameworks are closely coupled to a specific distributed system implementation [45], async framework [46] or programming language [47], [48]. As a notable exception to this, Antithesis [49] offers a proprietary cloud platform that runs a modified Linux kernel and emulates CPU instructions to provide system-wide determinism to userspace applications similar to FIZZLE. The approach taken by Antithesis provides stronger guarantees than FIZZLE (as raw system calls in an application are still deterministically handled), though FIZZLE’s pure-userspace solution enables performance optimizations in context switching and synchronization overhead that would not be otherwise possible. FIZZLE additionally offers compatibility with *nix operating systems and CPU architectures.

8. Conclusion

In this work, we evaluate the application of environment emulation and interposition to coverage-guided fuzzing and provide FIZZLE, a system implementing these properties. Our approach addresses existing challenges in fuzzing, namely when fuzzers are applied to multithreaded targets, nonreflexive protocols, forking applications and simultaneous fuzzing of multiple I/O inputs. By addressing these shortcomings, FIZZLE achieves 100% stability and bit-for-bit reproducible fuzzing runs even for complex, multithreaded servers. When applied to over a dozen target programs, our system additionally achieves enhanced coverage, stability, and execution speed when compared to established fuzzers. We use FIZZLE to discover three vulnerabilities in Open5GS, an open-source implementation of a 5G cellular core. FIZZLE [1] and the research artifacts of this paper [50] are made available and open-source for future research to build upon.

Acknowledgment

This work was supported in part by the National Science Foundation grants CNS-2055014 and CNS-2446321, and by the National Science Foundation Graduate Research Fellowship Program grant DGE-2236414. Any findings and opinions expressed in this material are those of the authors and do not necessarily reflect the views of the above funding agencies. Finally, the authors would like to thank the anonymous shepherd for their recommendations and the members of the Artifact Evaluation Committee for their feedback in improving our open-source artifacts.

References

- [1] N. Bennett, “Fizzle,” <https://github.com/determsim/fizzle>, 2025.
- [2] V.-T. Pham, M. Böhme, and A. Roychoudhury, “AFLNet: A greybox fuzzer for network protocols,” in *IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 460–465.
- [3] A. Andronidis and C. Cadar, “SnapFuzz: High-Throughput Fuzzing of Network Applications,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 340–351.
- [4] R. Swiecki, “Honggfuzz netdriver,” <https://github.com/google/honggfuzz/blob/master/docs/USAGE.md>, 2016.
- [5] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, “Stateful greybox fuzzing,” in *31st USENIX Security Symposium*. Boston, MA: USENIX Association, 2022, pp. 3255–3272. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/ba>
- [6] Y. Shoshitaishvili, “Preeny,” <https://github.com/zardus/preeny>, 2016.
- [7] S. Schumilo, C. Aschermann, A. Jemmett, A. Abbasi, and T. Holz, “Nyx-Net: Network fuzzing with incremental snapshots,” in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 166–180. [Online]. Available: <https://doi.org/10.1145/3492321.3519591>
- [8] J. Robben and M. Vanhoef, “Netfuzzlib: Adding first-class fuzzing support to network protocol implementations,” in *European Symposium on Research in Computer Security*, 2024.

- [9] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti, "LibAFL: A framework to build modular and reusable fuzzers," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1051–1065. [Online]. Available: <https://doi.org/10.1145/3548606.3560602>
- [10] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop On Offensive Technologies (WOOT 20)*, 2020.
- [11] N. Bars, M. Schloegel, N. Schiller, L. Bernhard, and T. Holz, "No peer, no cry: Network application fuzzing via fault injection," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 750–764.
- [12] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-Based Testing in Practice," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2024.
- [13] T. Liu, C. Curtsinger, and E. D. Berger, "Dthreads: Efficient deterministic multithreading," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, 2011, pp. 327–336.
- [14] W. Li, J. Ruan, G. Yi, L. Cheng, X. Luo, and H. Cai, "Poly-Fuzz: Holistic greybox fuzzing of multi-language systems," in *32nd USENIX Security Symposium*, 2023, pp. 1379–1396.
- [15] M. O'Neill, S. Heidbrink, J. Whitehead, T. Perdue, L. Dickinson, T. Collett, N. Bonner, K. Seamons, and D. Zappala, "The secure socket API: TLS as an operating system service," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 799–816.
- [16] D. Alden, "OpenBSD system-call pinning," <https://lwn.net/Articles/959562/>, 2024.
- [17] A. Inc, "Statically linked binaries on mac OS X," https://developer.apple.com/library/archive/qa/qa1118/_index.html, 2002.
- [18] A. Oppermann, "Information for 3rd party hardware driver vendors — FreeBSD," <https://wiki.freebsd.org/VendorInformation>, 2010.
- [19] F. Batschulat, "[port-solaris] sol11 AFS syscall," <https://www.cs.cmu.edu/afs/gco/archive/pipermail/port-solaris/2010-November/000009.html>, 2010.
- [20] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2123–2138.
- [21] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, and T. Holz, "SoK: Prudent evaluation practices for fuzzing," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 1974–1993.
- [22] R. Natella and V.-T. Pham, "ProFuzzBench: A benchmark for stateful protocol fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 662–665.
- [23] N. Bennett, W. Zhu, B. Simon, R. Kennedy, W. Enck, P. Traynor, and K. R. B. Butler, "RANSacked: A domain-informed approach for fuzzing LTE and 5G RAN-core interfaces," in *Proceedings of the 31st ACM Conference on Computer and Communications Security (CCS'24)*, 2024.
- [24] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of UNIX utilities," *Communications of the ACM*, vol. 33, no. 12, 1990.
- [25] M. Woo, S. K. Cha, S. Gottlieb, and D. Brumley, "Scheduling Black-Box Mutational Fuzzing," in *Proceedings of the ACM SIGSAC Conference on Computer & Communications Security (CCS)*, 2013.
- [26] C. Lyu, S. Ji, X. Zhang, H. Liang, B. Zhao, K. Lu, and R. Beyah, "EMS: History-driven mutation for coverage-based fuzzing," in *Proceedings of the ISOC Network and Distributed Systems Security (NDSS) Symposium*, 2022.
- [27] P. Godefroid, M. Y. Levin, D. A. Molnar et al., "Automated whitebox fuzz testing," in *Proceedings of the ISOC Network and Distributed Systems Security (NDSS) Symposium*, 2008.
- [28] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2005.
- [29] T. Blazytko, C. Aschermann, M. Schlögel, A. R. Abbasi, S. Schumilo, S. Wörner, and T. Holz, "GRIMOIRE: Synthesizing structure while fuzzing," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019.
- [30] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," *Proceedings of the ISOC Network and Distributed System Security (NDSS) Symposium*, 2019.
- [31] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks, "Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing," in *Proceedings of the USENIX Security Symposium*, 2021.
- [32] S. K. Cha, T. Avgerinos, A. Rebert, and D. Brumley, "Unleashing mayhem on binary code," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2012.
- [33] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A Survey for Roadmap," *ACM Computing Surveys (CSUR)*, vol. 54, no. 11s, 2022.
- [34] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, Nov. 2021.
- [35] G. Banks, M. Cova, V. Felmetser, K. Almeroth, R. Kemmerer, and G. Vigna, "SNOOZE: Toward a Stateful NetwOrk prOtocol fuzZEer," in *Information Security: 9th International Conference*. Springer, 2006, pp. 343–358.
- [36] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, "Prospex: Protocol specification extraction," in *2009 30th IEEE Symposium on Security and Privacy*. IEEE, 2009, pp. 110–125.
- [37] T. Kitagawa, M. Hanaoka, and K. Kono, "AspFuzz: A state-aware protocol fuzzer based on application-layer protocols," *The IEEE symposium on Computers and Communications*, pp. 202–208, 2010.
- [38] S. Gorbunov and A. Rosenbloom, "AutoFuzz: Automated network protocol fuzzing framework," *IJCSNS International Journal of Computer Science and Network Security*, vol. 10, no. 8, 2010. [Online]. Available: <https://api.semanticscholar.org/CorpusID:18430752>
- [39] J. Li, S. Li, G. Sun, T. Chen, and H. Yu, "SNPSFuzzer: A fast greybox fuzzer for stateful network protocols using snapshots," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2673–2687, 2022.
- [40] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, 2022.
- [41] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, and Y. Xiang, "Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2021, pp. 337–350.
- [42] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, and J. Sun, "Bleem: Packet sequence oriented fuzzing for protocol implementations," in *32nd USENIX Security Symposium*. Anaheim, CA: USENIX Association, 2023, pp. 4481–4498. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/luo-zhengxiong>
- [43] M. Eddington, "The peach project," <https://peachtech.gitlab.io/peach-fuzzer-community/>, 2008.
- [44] J. Pereyda, "Boofuzz: Network Protocol Fuzzing for Humans," <https://github.com/jtpereyda/boofuzz>, 2015.
- [45] A. Miller, "BUGGIFY," <https://transactional.blog/simulation/buggify>, 2021.
- [46] Tokio, "turmoil: Add hardship to your tests." <https://github.com/tokio-rs/turmoil>, 2025.

- [47] A. S. Marques, “(mostly) deterministic simulation testing in go,” <https://www.polarsignals.com/blog/posts/2024/05/28/mostly-dst-in-go>, 2024.
- [48] J. Wangrun, “madsim: Magical deterministic simulator for distributed systems in Rust,” <https://github.com/madsim-rs/madsim>, 2025.
- [49] “Antithesis,” <https://antithesis.com/>, 2025.
- [50] N. Bennett, “Fizzle (artifact),” <https://github.com/determsim/fizzle>, 2025.

Appendix A.

Deterministic Multithreading Algorithm

Algorithm 1 Deterministic Thread Control Flow Tracking

```

1:  $S : \text{Map}()$  ▷ Per-thread semaphores
2:  $Q : \text{Queue}()$  ▷ Threads ready to be executed
3: function PTHREAD_CREATE_HOOK( $f, arg$ )
4:    $push(Q, self)$ 
5:    $t \leftarrow pthread\_create(\text{THREAD\_WRAPPER}, (f, arg))$ 
6:    $sem\_wait(S, self)$ 
7: function THREAD_WRAPPER( $tup$ )
8:    $f, arg \leftarrow tup$ 
9:    $insert(S, self)$ 
10:   $f(arg)$ 
11:  PTHREAD_EXIT_HOOK()
12: function PTHREAD_EXIT_HOOK( $\cdot$ )
13:   $remove(S, self)$ 
14:   $t \leftarrow pop(Q)$ 
15:  if  $t = nil$  then
16:    Move on to next fuzzing round
17:  else
18:     $sem\_post(S, t)$ 
19:     $pthread\_exit()$ 
20: function SYSTEM_LIBRARY_HOOK( $args$ )
21:  Simulate system library function with  $args$ 
22:  if simulation unblocks any thread  $t$  then
23:     $push(Q, t)$ 
24:  if simulation would block  $self$  then
25:     $t \leftarrow pop(Q)$ 
26:    if  $t = nil$  then
27:      Move on to next fuzzing round
28:    else
29:       $sem\_post(S, t)$ 
30:       $sem\_wait(S, self)$ 

```

Algorithm 1 offers a high-level description of routines used by FIZZLE to handle multithreading deterministically. Two global data structures, C and Q , are respectively used to store per-thread semaphores and track what threads are or ready to be executed. The PTHREAD_CREATE_HOOK and PTHREAD_EXIT_HOOK routines serve as hooks their respective pthread functions, while GENERIC_LIBC_HOOK shims any other libc functions. To clarify the purpose of these routines, we hereafter describe their effects within the context of a simple multithreaded program being fuzzed.

At the beginning of a fuzzing round, all threads are marked as ready to be executed (i.e., $Q \leftarrow keys(C)$), and fuzzing input is passed to the appropriate simulated socket(s). A single thread is then started; it executes until it

reaches a libc method that it would block on (line 24 of Algorithm 1). It then posts to the semaphore for the next ready thread in Q and awaits its own semaphore (lines 29-30). If at any point a thread calls a libc function that would unblock execution of another thread, such as *write* or *pthread_mutex_unlock*, it adds that thread to Q (lines 22-23). Likewise, if at any point a thread creates another thread, it immediately suspends execution so that the new thread can insert its thread ID/semaphore pair into C and run (lines 3-11). Terminating threads remove their thread ID/semaphore from C and awaken the next thread in Q prior to exit (lines 12-19).

This algorithm leverages semaphores to provide three assurances regarding multithreaded behavior. First, transitions from one thread to another are designed to guarantee that application code is never executed concurrently on multiple threads during the transition. Second, thread creation/termination routines ensure that C has up-to-date semaphores for each thread prior to any other libc functions being called. Third, thread creation/termination routines are handled similarly as transitions from one thread to another, such that application code is never executed concurrently.

Appendix B. Meta-Review

B.1. Summary

The paper presents Fizzle, a high-performance, deterministic simulation testing framework for fuzzing network applications. After outlining key challenges in fuzzing such applications such as non-determinism from multi-threading, complex network interactions, and reliance on randomness, the paper introduces an approach that turns fuzzing into a deterministic process. Fizzle interposes on system library calls to enforce sequential execution of multi-threaded code, emulate intricate network behaviors, and precisely control sources of randomness. By tracking synchronization and I/O primitives, it can accurately determine when each input has been completely processed, thereby eliminating the need for timeouts or manual annotations. A full prototype of Fizzle is evaluated on ProFuzzBench and a few other targets. The results show that Fizzle achieves coverage comparable to or better than existing fuzzers in less time and uncovers three previously unknown vulnerabilities in Open5GS.

B.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Addresses a Long-Known Issue
- Provides a Valuable Step Forward in an Established Field

B.3. Reasons for Acceptance

- 1) Addresses nondeterminism in fuzzing: Fizzle provides a practical solution to a long-standing challenge, enabling reproducibility of fuzzing results for complex, non-deterministic targets such as network servers.
- 2) Well-integrated deterministic framework: Fizzle enforces sequential execution and controls I/O, timers, threads, and randomness in a unified way, combining previously scattered ideas into a coherent system.
- 3) Ease of use and extensibility: Fizzle works transparently with unmodified targets, requires little to no source code changes, and can be extended to handle additional environmental inputs (e.g., DNS responses).
- 4) Empirical evaluation: The authors built a full prototype of the proposed approach and the empirical evaluation demonstrates that Fizzle has a better performance compared to existing fuzzers, including faster coverage and reproducible results. The tool will be published.

B.4. Noteworthy Concerns

- 1) The evaluation is limited and could be strengthened by: (1) selecting a more diverse set of heterogeneous targets; (2) running experiments on different platforms; and (3) conducting focused experiments to validate the determinization guarantees of Fizzle.
- 2) Fizzle was only able to find new vulnerabilities in one target.